

Split-Forest Normal Forms and Finite Abstractions for $\mathcal{ALCI}_{\text{RCC5}}$

Organized Around the Three Core Theorems

Consolidated proof manuscript

May 30, 2026

Abstract

This paper isolates the three load-bearing parts of the split-forest decidability argument for $\mathcal{ALCI}$ with the five RCC5 atomic roles EQ, PP, PPI, PO, DR, over abstract complete atomic RCC5 frames with strong equality. The point is to separate the model-theoretic normal form from the finite representation theorem and from the automata decision layer. The three central results are: (A) every satisfiable concept has a witness-generated saturated split-forest presentation; (B) saturated split forests are exactly captured, up to satisfiability of all closure formulas, by finite frontier/pair/triple/equality-port abstractions; and (C) a finite alternating parity tree automaton accepts exactly the valid finite abstractions. The paper also explains what remains of the original split-forest idea: the split forest is retained as the semantic normal form, while the automaton is used only as the regularity and eventuality engine.

Contents

1	Purpose and relation to the original split-forest idea	1
2	Syntax and abstract RCC5 semantics	2
2.1	Concept language	2
2.2	RCC5 relation algebra	3
3	Theorem A: split-forest normal form	4
3.1	Witness thinning	4
3.2	Generated containment covers and split copies	4
3.3	Side contexts and forced verticalization	5
3.4	Saturated split forests	6
4	Theorem B: finite abstraction adequacy	7
4.1	Frontier profiles	7
4.2	Pair and triple product states	8
4.3	Finite abstraction objects	9
4.4	Extraction and realization	10
5	Theorem C: automaton correctness	11
5.1	Automaton model	11
5.2	Transition conditions	11
5.3	Parity condition	12
5.4	Correctness of the automaton	12

6	Where the proof obligations live	13
7	Finite bounds and effectivity	13
8	Concluding formulation	14
A	Detailed side-context saturation argument	14
B	Exact equality-port quotient	15
C	Pair and triple exhaustion	16
D	Automaton finiteness and emptiness	17
E	Suggested division between main paper and	17

1 Purpose and relation to the original split-forest idea

The original split-forest proposal was not merely a notational device. It contained the main semantic insight: arbitrary RCC5/Kripke models should first be replaced by sparse witness-generated presentations in which containment is represented vertically and non-containment interaction is represented by finite side interfaces. Once this normal form is available, decidability becomes a finite search problem over regular labelled trees.

Several rounds of repair showed that the split-forest idea and the decision method should not be forced to do the same job. The split forest should answer the model-theoretic question

what kind of models suffice?

whereas the automaton should answer the algorithmic question

is there a regular object of that kind?

This paper therefore organizes the argument around the following three theorems.

Theorem A. Every satisfiable concept has a witness-generated saturated split-forest presentation.

Theorem B. The finite frontier/pair/triple/equality-port alphabet is sound and complete for saturated split forests.

Theorem C. The parity tree automaton accepts exactly the finite abstractions of satisfying saturated split forests.

The table summarizes the final status of the original proof components.

Original component	Final role
Witness thinning	Retained. It reduces arbitrary abstract models to sparse witness-generated models.
Generated containment cover	Retained. It is a generated cover presentation, not the semantic Hasse diagram of an arbitrary starting model.
Split copies	Retained. They handle elements with several selected incomparable superparts.
Sibling/side interfaces	Retained but strengthened to saturated frontier profiles with forced verticalization.
Residual DR/PO frontier	Retained only after saturation has removed all forced PP, PPI, EQ cross-pairs.
Typed equality	Retained as exact equality ports and a typed congruence.
Manual blocking/request cycles	Replaced by parity acceptance over finite request states.
Manual certificate enumeration	Replaced by emptiness of a finite alternating parity tree automaton.
Patchwork intuition	Formalized by finite pair and triple product-state representatives.

Thus the split-forest model class is not technical debt. It is the normal form that makes a tree automaton applicable. What changed is that the finite abstraction and eventuality parts are no longer hand-managed by ad hoc cyclic certificate clauses; they are represented as a finite alphabet and checked by the automaton.

2 Syntax and abstract RCC5 semantics

2.1 Concept language

Let

$$\text{Rel} = \{\text{EQ}, \text{PP}, \text{PPI}, \text{PO}, \text{DR}\}.$$

The converse operation is

$$\text{inv}(\text{EQ}) = \text{EQ}, \quad \text{inv}(\text{PP}) = \text{PPI}, \quad \text{inv}(\text{PPI}) = \text{PP}, \quad \text{inv}(\text{PO}) = \text{PO}, \quad \text{inv}(\text{DR}) = \text{DR}.$$

Concepts are built from concept names, Boolean connectives, and restrictions

$$\exists R.C, \quad \forall R.C, \quad R \in \text{Rel}.$$

All concepts are assumed to be in negation normal form. The NNF complement of C is denoted $\overline{C}$ and satisfies the usual identities

$$\begin{aligned} \overline{\overline{A}} &= A, & \overline{\neg A} &= A, \\ \overline{C \sqcap D} &= \overline{C} \sqcup \overline{D}, & \overline{C \sqcup D} &= \overline{C} \sqcap \overline{D}, \\ \overline{\exists R.C} &= \forall R.\overline{C}, & \overline{\forall R.C} &= \exists R.\overline{C}. \end{aligned}$$

Definition 2.1 (Closure). *For an input concept C_0 , $\text{Cl}(C_0)$ is the least finite set containing C_0 and closed under subformulas and NNF complement. Equality restrictions may either be eliminated by the strong-equality identities*

$$\exists \text{EQ}.C \equiv C, \quad \forall \text{EQ}.C \equiv C,$$

or retained with these equivalences imposed on types.

Definition 2.2 (Types). A type over $\text{Cl}(C_0)$ is a subset $t \subseteq \text{Cl}(C_0)$ satisfying:

(T1) for every $D \in \text{Cl}(C_0)$, exactly one of $D, \bar{D}$ belongs to t ;

(T2) $D \sqcap E \in t$ iff $D \in t$ and $E \in t$;

(T3) $D \sqcup E \in t$ iff $D \in t$ or $E \in t$;

(T4) $\exists \text{EQ}.D \in t$ iff $D \in t$;

(T5) $\forall \text{EQ}.D \in t$ iff $D \in t$.

Let $\text{Type}(C_0)$ be the finite set of such types.

2.2 RCC5 relation algebra

The RCC5 atomic composition table used throughout is the following relation-valued operation $\text{Comp}(R, S) \subseteq \text{Rel}$:

Comp	EQ	PP	PPI	PO	DR
EQ	{EQ}	{PP}	{PPI}	{PO}	{DR}
PP	{PP}	{PP}	Rel	{PP, PO, DR}	{DR}
PPI	{PPI}	{EQ, PP, PPI, PO}	{PPI}	{PPI, PO}	{PPI, PO, DR}
PO	{PO}	{PP, PO}	{PPI, PO, DR}	Rel	{PPI, PO, DR}
DR	{DR}	{PP, PO, DR}	{DR}	{PP, PO, DR}	Rel

Definition 2.3 (Abstract strong RCC5 frame). An abstract strong RCC5 frame is a nonempty set Δ together with a map

$$\rho : \Delta^2 \rightarrow \text{Rel}$$

such that for all $a, b, c \in \Delta$:

(R1) $\rho(a, b) = \text{EQ}$ iff $a = b$;

(R2) $\rho(b, a) = \text{inv}(\rho(a, b))$;

(R3) $\rho(a, c) \in \text{Comp}(\rho(a, b), \rho(b, c))$.

Definition 2.4 (Interpretations). An interpretation $\mathcal{I}$ consists of a strong RCC5 frame $(\Delta^{\mathcal{I}}, \rho^{\mathcal{I}})$ and an interpretation of each concept name as a subset of $\Delta^{\mathcal{I}}$. Boolean clauses are standard, and

$$a \models_{\mathcal{I}} \exists R.C \iff \exists b \in \Delta^{\mathcal{I}} [\rho^{\mathcal{I}}(a, b) = R \text{ and } b \models_{\mathcal{I}} C],$$

$$a \models_{\mathcal{I}} \forall R.C \iff \forall b \in \Delta^{\mathcal{I}} [\rho^{\mathcal{I}}(a, b) = R \Rightarrow b \models_{\mathcal{I}} C].$$

A concept C_0 is satisfiable if $a \models_{\mathcal{I}} C_0$ for some interpretation $\mathcal{I}$ and $a \in \Delta^{\mathcal{I}}$.

3 Theorem A: split-forest normal form

Theorem A is the semantic normalization theorem. It has three stages: witness thinning, generated containment covers, and saturation of side interfaces.

3.1 Witness thinning

Definition 3.1 (Chosen witnesses). *Let $\mathcal{I}, a_0 \models C_0$. For every $a \in \Delta^{\mathcal{I}}$ and every existential $\exists R.D \in \text{Cl}(C_0)$ with $a \models_{\mathcal{I}} \exists R.D$, choose one witness*

$$w(a, \exists R.D)$$

such that

$$\rho^{\mathcal{I}}(a, w(a, \exists R.D)) = R, \quad w(a, \exists R.D) \models_{\mathcal{I}} D.$$

Definition 3.2 (Witness-generated subdomain). *Let $W_0 = \{a_0\}$. Having defined W_n , let W_{n+1} contain W_n and, for every $a \in W_n$ and every $\exists R.D \in \text{Cl}(C_0)$ true at a , the chosen witness $w(a, \exists R.D)$. Put*

$$W = \bigcup_{n < \omega} W_n.$$

Let $\mathcal{I} \upharpoonright W$ be the induced subinterpretation on W .

Lemma 3.3 (Witness thinning). *For every $D \in \text{Cl}(C_0)$ and every $a \in W$,*

$$a \models_{\mathcal{I}} D \iff a \models_{\mathcal{I} \upharpoonright W} D.$$

In particular, $a_0 \models_{\mathcal{I} \upharpoonright W} C_0$.

Proof. The proof is by structural induction on D and uses closure under NNF complement. Atomic and Boolean cases are immediate because concept-name interpretations are restricted to W . For $D = \exists R.E$, if $a \models_{\mathcal{I}} \exists R.E$, the selected witness belongs to W by construction and satisfies E by the induction hypothesis. The reverse direction is immediate because $\mathcal{I} \upharpoonright W$ is an induced subinterpretation. For $D = \forall R.E$, the forward direction holds because every R -successor in W is also an R -successor in $\mathcal{I}$. For the reverse direction, suppose $a \not\models_{\mathcal{I}} \forall R.E$. Then $a \models_{\mathcal{I}} \exists R.\bar{E}$, and since $\exists R.\bar{E} \in \text{Cl}(C_0)$, its selected witness is in W and satisfies $\bar{E}$ by the induction hypothesis. Hence $a \not\models_{\mathcal{I} \upharpoonright W} \forall R.E$. $\square$

The thinning lemma is the formal replacement for any assumption about semantic Hasse diagrams of arbitrary starting models. The resulting model contains only the root and selected witnesses. Generated cover edges are chosen witness links, not necessarily immediate covers in the original semantic order.

3.2 Generated containment covers and split copies

In the containment-oriented presentation, a vertical parent is a selected proper superpart. Thus along one branch,

$$x\text{PP}y \quad \text{iff} \quad y \text{ is a strict generated ancestor of } x,$$

and conversely

$$x\text{PPI}y \quad \text{iff} \quad y \text{ is a strict generated descendant of } x.$$

The generated edge relation is auxiliary. Semantic PP/PPI is its strict vertical transitive closure plus the explicitly recorded nonvertical labels in side interfaces.

A single occurrence cannot always carry all selected superparts of a semantic element. If x has two selected PP-witnesses y and z that must be incomparable, then one tree occurrence of x cannot lie below both y and z in a tree. The split-forest presentation therefore permits multiple occurrences of the same semantic element, connected by equality ports and later quotiented by a typed equality congruence.

Definition 3.4 (Weak split occurrence). *A weak split occurrence is a node u equipped with:*

- *an origin element $\text{orig}(u) \in W$;*
- *a type $\text{tp}(u) = \{D \in \text{Cl}(C_0) : \text{orig}(u) \models D\}$;*
- *a finite list of equality ports naming other occurrences with the same origin, subject to exact equality congruence conditions below.*

Two occurrences may have the same origin without being identified by blocking. Blocking/profile repetition and semantic equality are separate relations:

$$\equiv_{\text{blk}} \neq \equiv_{\text{EQ}} .$$

3.3 Side contexts and forced verticalization

Side contexts represent selected DR/PO witnesses and nonvertical interactions. The important repair is that side contexts are not mere bags of side witnesses. They are saturated finite local diagrams.

Definition 3.5 (Ranked side context). *Fix a computable rank bound $h = h(C_0)$, for instance the number of modal subformulas in $\text{Cl}(C_0)$ plus the number of types. A ranked side context for a source occurrence u is a finite labelled diagram $S(u)$ of rank at most h containing:*

- (S1) every selected DR/PO witness of u required by an existential in $\text{Cl}(C_0)$;*
- (S2) the relevant vertical boundary of u : its selected parent, selected children, equality mates, and bounded-depth vertical threshold nodes needed for forced verticalization;*
- (S3) all inherited labels among context positions and between context positions and the boundary;*
- (S4) equality-port declarations for every inherited EQ label;*
- (S5) local vertical strata for every inherited PP or PPI side/tree or side/side pair;*
- (S6) recursively, lower-rank side contexts required by selected existential formulas at newly introduced context positions.*

The recursion is well founded because the rank strictly decreases when a side context calls a subordinate side context.

Definition 3.6 (Forced verticalization). *Let w be introduced as a DR or PO side witness for a source u . Before w may be placed into the residual frontier, the construction checks the inherited relation between w and every relevant vertical boundary occurrence b of u .*

- *If $\rho(b, w) = \text{EQ}$, then w is represented by an equality port.*
- *If $\rho(b, w) = \text{PP}$ or $\rho(b, w) = \text{PPI}$, then an occurrence or equality mate of w is spliced into a local vertical stratum at the appropriate bounded threshold.*
- *Only if no such EQ, PP, PPI interaction remains is the pair classified as residual, and then its label is DR or PO.*

The same check is applied to side/side pairs and to both ancestor and descendant tails. Thus an infinite ancestor PPI tail and an infinite descendant PP tail are handled symmetrically by vertical reachability, not by an impossible bounded side context.

Lemma 3.7 (Residual-frontier lemma). *After ranked side-context saturation and forced verticalization, every residual pair whose endpoints are not equality-linked, not vertically comparable, and not inside an explicit side-interface stratum has label in $\{\text{DR}, \text{PO}\}$.*

Proof. Let (p, q) be such a residual pair. During saturation, every pair involving a newly introduced side position and every relevant vertical or side position is inspected. If its inherited label is EQ, the pair is removed from the residual frontier and represented by an equality port. If its inherited label is PP or PPI, the pair is removed from the residual frontier and represented by a local vertical stratum. The only RCC5 base labels left after these three cases are DR and PO. Hence every residual pair has one of those two labels. $\square$

3.4 Saturated split forests

Definition 3.8 (Saturated split-forest presentation). *A saturated split-forest presentation for C_0 consists of:*

- (F1) *a possibly infinite generated forest of occurrences, with vertical parent/child links interpreted as generated containment covers;*
- (F2) *a type $\text{tp}(u) \in \text{Type}(C_0)$ at every occurrence;*
- (F3) *exact equality ports $\equiv_{\text{EQ}}$ between weak split occurrences, disjoint from blocking/profile repetition;*
- (F4) *ranked saturated side contexts at occurrences;*
- (F5) *a total RCC5 label for every pair of occurrences, obtained from vertical closure, equality ports, side contexts, and residual DR/PO labels;*
- (F6) *for every existential $\exists R.D \in \text{tp}(u)$, an explicit support: a vertical reachable occurrence, an equality-port mate's vertical reachable occurrence, or a side-context occurrence labelled by R and carrying D ;*
- (F7) *for every universal $\forall R.D \in \text{tp}(u)$, propagation of D to every represented R -target, including targets generated by RCC5 composition through side contexts and vertical paths.*

It is strong if quotienting by $\equiv_{\text{EQ}}$ yields a strong RCC5 interpretation with EQ iff identity.

Theorem 3.9 (Theorem A: split-forest normal form). *If C_0 is satisfiable over abstract strong RCC5 frames, then C_0 has a witness-generated saturated split-forest presentation.*

Proof. Start with a model $\mathcal{I}, a_0 \models C_0$ and apply witness thinning. The thinned model is generated from a_0 by selected witnesses for existential formulas in $\text{Cl}(C_0)$, and truth of all closure formulas is preserved.

For selected PPI witnesses, create generated child occurrences. For selected PP witnesses, create generated parent occurrences. If one semantic element requires several incomparable selected superparts, create several weak split occurrences of that element, one under each selected superpart, and connect them by equality ports. These ports are semantic equality data; they are not blocking identifications.

For selected DR/PO witnesses, place the witness into the ranked side context of the source. Saturate the context by repeatedly adding inherited labels to the vertical boundary and to other side positions. Whenever the inherited label is EQ, PP, or PPI, remove the pair from the residual frontier and represent it by an equality port or local vertical stratum. Because subordinate side contexts have strictly smaller rank, the saturation terminates within the computable bound $h(C_0)$. The residual-frontier lemma then ensures that residual frontier labels are only DR or PO.

Define the RCC5 label between any two occurrences by case distinction on their nearest common represented interface: vertical ancestor/descendant relation, equality port, side-context matrix, or residual frontier. The label agrees with the inherited label from the thinned model for all selected supports and all saturated boundary positions. RCC5 converse and composition are inherited locally from the original model and enforced at context overlaps; the finite abstraction theorem below makes this overlap enforcement explicit. Existential supports are present by construction, and universal restrictions are preserved because every generated or composition-forced target appears in a represented vertical, equality, or side-interface class. Quotienting exact equality ports gives strong equality. $\square$

4 Theorem B: finite abstraction adequacy

Theorem B is the most delicate part. It states that saturated split forests can be represented by a finite alphabet of frontier profiles, pair states, triple states, and equality ports; conversely, every valid finite abstraction unfolds to a genuine saturated split forest.

4.1 Frontier profiles

Definition 4.1 (Local positions). *For fixed C_0 , let $K = K(C_0)$ be a computable bound on the number of live positions needed at one interface. A live position is one of:*

- *the current occurrence;*
- *one of finitely many equality-port mates;*
- *a selected parent or child slot;*
- *a side-context position of bounded rank;*
- *a boundary threshold position used for forced verticalization.*

Let $\text{Pos}(C_0)$ be a finite set of abstract names for these possible live positions.

Definition 4.2 (Frontier profile). *A frontier profile over C_0 is a finite object*

$$P = (L, \tau, E, M, U, X, Y)$$

where:

- (P1) $L \subseteq \text{Pos}(C_0)$ *is the live position set;*
- (P2) $\tau : L \rightarrow \text{Type}(C_0)$ *assigns types;*
- (P3) E *is an exact equality-port equivalence relation on L ;*
- (P4) $M : L^2 \rightarrow \text{Rel}$ *is a relation matrix satisfying $M(p, q) = \text{EQ}$ iff $p E q$, $M(q, p) = \text{inv}(M(p, q))$, and the RCC5 composition table on all triples in L ;*

- (P5) U is a set of universal obligations of the form (p, R, D) meaning that every represented R -target of p must carry D ;
- (P6) X is a set of existential support declarations of the form $(p, \exists R.D, s)$, where s is a local support position, an equality-port transfer, a vertical reachability request, or a side-context occurrence;
- (P7) Y is a finite side-context descriptor, including recursive lower-rank side descriptors, forced verticalization records, and residual DR/PO frontier labels.

The profile is legal if these components satisfy the local type, equality, relation, side-context, existential, and universal consistency clauses listed below.

Definition 4.3 (Local legality clauses). *A frontier profile $P = (L, \tau, E, M, U, X, Y)$ is locally legal if:*

- (L1) *type clauses (T1)–(T5) hold at each $p \in L$;*
- (L2) *E is an equivalence relation and $M(p, q) = \text{EQ}$ iff $p E q$;*
- (L3) *if $p E q$, then $\tau(p) = \tau(q)$ and p, q have the same externally visible equality ports, universal obligations, and side-interface behaviour;*
- (L4) *for all $p, q, r \in L$, $M(p, r) \in \text{Comp}(M(p, q), M(q, r))$;*
- (L5) *if $\exists R.D \in \tau(p)$, then X contains a support declaration whose represented target has relation R from p and carries D ;*
- (L6) *if $\forall R.D \in \tau(p)$, then $(p, R, D) \in U$ and D is propagated to every locally represented R -target of p ;*
- (L7) *side-context descriptor Y is ranked, saturated, and obeys forced verticalization: every side/tree and side/side pair labelled EQ, PP, or PPI is represented by an equality port or local vertical stratum before residual classification;*
- (L8) *every residual side-frontier pair in Y is labelled DR or PO.*

4.2 Pair and triple product states

A finite frontier profile describes only one interface. The unfolded split forest contains pairs and triples whose endpoints may lie in different subtrees. The repaired abstraction therefore records finite product-state representatives for such pairs and triples.

Definition 4.4 (Pair state). *A pair state is a finite record*

$$\pi = (P, Q, i, j, R, \eta)$$

where P, Q are frontier profiles, $i \in P$, $j \in Q$ are endpoint positions, $R \in \text{Rel}$ is the represented RCC5 label, and η is a finite interface trace describing how the pair crosses the least common represented interface: same local profile, ancestor/descendant path, equality-port transfer, side-context transfer, or residual frontier transfer. The set of pair states is denoted $\text{Pair}(C_0)$.

Definition 4.5 (Triple state). *A triple state is a compatible ordered triple of pair states*

$$\theta = (\pi_{12}, \pi_{23}, \pi_{13})$$

with endpoint profiles and positions matching on overlaps and with

$$R_{13} \in \text{Comp}(R_{12}, R_{23}),$$

where R_{ab} is the RCC5 label recorded by π_{ab} . The set of triple states is denoted $\text{Trip}(C_0)$.

Remark 4.6 (Why pair/triple states are needed). *It is not enough to say that every global triple is covered by some unspecified patch. Universal restrictions and RCC5 composition can create targets that are not selected witnesses. Pair states give a finite representative for every pair of occurrences, and triple states certify that the three pair representatives chosen for any triple compose correctly.*

Definition 4.7 (Product-state transition). *Given a parent profile and the profiles of its finitely many children, a product-state transition specifies:*

- (PS1) how every local pair inside one profile maps to a pair state;*
- (PS2) how every cross-child pair is mapped through the parent frontier or a saturated side-context descriptor;*
- (PS3) how ancestor/descendant pairs compose through the vertical containment relation;*
- (PS4) how equality-port transfers are resolved without identifying blocking copies;*
- (PS5) how every compatible ordered triple of generated pair states maps to a triple state in $\text{Trip}(C_0)$.*

The transition is legal only if all generated pair and triple states are in $\text{Pair}(C_0)$ and $\text{Trip}(C_0)$ and all universal obligations are propagated along pair states with the required relation label.

4.3 Finite abstraction objects

Definition 4.8 (Finite split-forest abstraction). *A finite abstraction for C_0 is a finite ranked tree alphabet $\Sigma(C_0)$ whose letters are locally legal frontier profiles, together with finite pair and triple transition tables*

$$\delta_{\text{Pair}}, \delta_{\text{Trip}}$$

assigning product-state representatives to every parent/child interface combination. It is valid if:

- (A1) the root profile contains a distinguished position r with $C_0 \in \tau(r)$;*
- (A2) every local profile is legal;*
- (A3) every parent/child tuple respects the generated containment orientation;*
- (A4) every selected PP or PPI support is represented by vertical reachability, possibly after equality-port transfer to a split mate;*
- (A5) every selected DR or PO support is represented inside a saturated side context;*
- (A6) every equality port is exact: equality-port equivalent positions have identical type and external interface data, and no strict vertical path connects equality-port mates;*

- (A7) every pair in every finite prefix of the unfolding has a pair-state representative generated by δ_{Pair} ;
- (A8) every ordered triple in every finite prefix has a triple-state representative generated by δ_{Trip} ;
- (A9) universal obligations are propagated to all pair-state represented R -targets;
- (A10) transitive existential requests for PP and PPI are recorded as finite eventuality obligations for the automaton.

The clauses are finite because $\text{Type}(C_0)$, $\text{Pos}(C_0)$, side-context rank, equality-port names, pair states, and triple states are all bounded by computable functions of $|\text{Cl}(C_0)|$.

4.4 Extraction and realization

Lemma 4.9 (Extraction of a finite abstraction). *Every saturated split-forest presentation of C_0 induces a valid finite abstraction.*

Proof. Take the type of each occurrence from the presentation. At each occurrence, collect its bounded live interface: equality mates, selected vertical boundary positions, saturated side-context positions, and threshold positions. Since the interface size and side-context rank are bounded by functions of C_0 , there are only finitely many isomorphism types. These are the frontier profiles.

For every pair of occurrences in a finite prefix, compute its nearest represented interface: the same local profile, an ancestor/descendant vertical segment, an equality-port transfer, a saturated side context, or a residual frontier. Record the corresponding pair state. Because all ingredients are chosen from finite profile and context alphabets, only finitely many pair states occur. For every triple, record the three induced pair states and the inherited RCC5 composition witness. This produces a triple state, and compatibility follows from the original RCC5 frame.

Existential support declarations are extracted from the selected witness supports of the saturated presentation. Universal obligations are extracted from the truth of universal formulas and from the complete pair-state coverage of represented targets. Equality-port exactness is inherited from origin equality in the presentation. Thus all validity clauses hold. $\square$

Lemma 4.10 (Realization from a valid finite abstraction). *Every valid finite abstraction unfolds to a saturated split-forest presentation satisfying all closure formulas recorded in its types.*

Proof. Unfold the finite tree alphabet into a possibly infinite labelled tree. Each node occurrence receives the type, side-context positions, equality ports, and relation matrix specified by its frontier profile. Vertical parent/child links supply generated containment covers. Equality-port declarations induce an equivalence relation $\equiv_{\text{EQ}}$ on occurrences. Blocking/profile repetition is not part of $\equiv_{\text{EQ}}$.

Define the RCC5 label between any two occurrences by the pair state generated by repeated use of δ_{Pair} from the least common represented interface. This is well defined by the overlap compatibility component of pair states. Define labels for triples through δ_{Trip} . Since every triple state satisfies the RCC5 composition table, the global labelling satisfies composition for all triples of occurrences. Converse and strong equality are enforced by the pair-state and equality-port clauses.

The interpretation of concept names is given by types. The truth lemma for all $D \in \text{Cl}(C_0)$ is proved by induction on D . Boolean cases follow from type legality. For $\exists R.D$, the abstraction contains a support declaration. If the support is local or side-contextual, the required target exists in the unfolding by construction. If it is vertical, the automaton/eventuality layer guarantees a strict reachable target carrying D . If it is equality-transfer vertical, first move to the equality-port mate and then follow the vertical support; after quotienting exact equality ports, this is a valid

support for the original occurrence. For $\forall R.D$, pair-state exhaustion says every R -target has a pair representative, and universal propagation requires D in the target type. The induction hypothesis finishes the proof.

Finally quotient by $\equiv_{\text{EQ}}$. Exact equality ports guarantee that relation labels, types, supports, and universal obligations are congruent across representatives. Therefore the quotient is a strong RCC5 interpretation. $\square$

Theorem 4.11 (Theorem B: finite abstraction adequacy). *For an input concept C_0 , saturated split-forest presentations satisfying C_0 exist iff valid finite abstractions for C_0 exist.*

Proof. The forward direction is the extraction lemma. The reverse direction is the realization lemma. Both constructions preserve the root type and hence preserve satisfaction of C_0 . $\square$

5 Theorem C: automaton correctness

Theorem C says that valid finite abstractions are precisely the objects recognized by an effectively constructible parity tree automaton.

5.1 Automaton model

We use finite alternating parity tree automata over ranked trees. The branching rank is bounded by a computable number $B(C_0)$ sufficient for all selected vertical children, equality-port copies, side-context roots, and dummy padding children. Alternation is convenient for universal propagation and for spawning request monitors. Nothing in the construction requires an infinite-state device.

Definition 5.1 (Automaton alphabet). *The input alphabet $\Sigma(C_0)$ consists of all locally legal frontier profiles over C_0 , including side-context descriptors, equality ports, pair-state interface tables, and triple-state transition summaries. Since all components are finite and effectively enumerable, $\Sigma(C_0)$ is finite and computable.*

Definition 5.2 (States). *The automaton $\mathcal{A}_{C_0}$ has the following finite classes of states:*

- (Q1) a main state q_{main} checking local profile legality and child-transition compatibility;
- (Q2) pair-checking states $q_{\text{Pair}}(\pi)$ for $\pi \in \text{Pair}(C_0)$;
- (Q3) triple-checking states $q_{\text{Trip}}(\theta)$ for $\theta \in \text{Trip}(C_0)$;
- (Q4) equality-port congruence states $q_{\text{EQ}}(e)$ for equality-port data e ;
- (Q5) universal-propagation states $q_{\forall}(R, D)$;
- (Q6) eventuality states $q_{\exists}^{\text{PP}, D}$ and $q_{\exists}^{\text{PPI}, D}$ for transitive vertical existential requests.

5.2 Transition conditions

At a node labelled by profile P , the automaton checks the following finite conditions.

- (C1) P is locally legal.
- (C2) The child profile tuple is compatible with the generated containment orientation and with the declared side-context ranks.

- (C3) Every local pair and cross-child pair has the declared pair-state representative.
- (C4) Every local triple and every triple formed across child interfaces has the declared triple-state representative.
- (C5) Equality ports are exact and congruent: equal ports have identical types and identical external pair/triple behaviour.
- (C6) Every existential formula has a declared support. Nonvertical DR/PO supports must appear in saturated side contexts. Vertical PP/PPI supports either appear immediately in the live boundary or create an eventuality state.
- (C7) Every universal formula $\forall R.D$ is propagated to all represented R -targets using pair states, including composition-generated targets and side-context targets.
- (C8) Every side-context residual frontier label is DR or PO after saturation and forced verticalization.

All checks are finite table lookups in $\Sigma(C_0)$, $\text{Pair}(C_0)$, and $\text{Trip}(C_0)$.

5.3 Parity condition

The only genuinely infinite obligations are transitive PP/PPI existential requests along vertical branches. The automaton handles them by parity acceptance.

For a request $\exists \text{PP}.D$ at an occurrence, the monitor follows the ancestor direction, allowing equality-port transfer to an exact split mate when declared. The request is discharged when a strict represented PP-target with D in its type is encountered. Dually, $\exists \text{PPI}.D$ follows the descendant direction. Pending request states receive odd priorities; discharged or inactive states receive smaller even priorities. The standard parity condition therefore enforces that no request remains pending forever.

Lemma 5.3 (Eventuality correctness). *Along every branch of an accepting run, every transitive PP or PPI existential request opened by a type is eventually discharged at a strict target of the required relation and type. Conversely, if all such requests are discharged in the represented split forest, the parity condition can be satisfied.*

Proof. The automaton has finitely many request kinds (R, D) with $R \in \{\text{PP}, \text{PPI}\}$ and $D \in \text{Cl}(C_0)$. While a request is pending, the run visits a state with the corresponding odd priority. When a strict target with D is found, the request monitor exits to an even-priority discharged state. Parity acceptance excludes runs in which some odd pending priority is seen infinitely often without a smaller even discharge priority recurring. Conversely, if every request has a strict target, choose the transition that follows the support path to the target and then discharges. Finiteness of the request set gives a uniform parity index. $\square$

5.4 Correctness of the automaton

Theorem 5.4 (Theorem C: automaton correctness). *For every concept C_0 ,*

$$L(\mathcal{A}_{C_0}) \neq \emptyset \iff C_0 \text{ has a valid finite split-forest abstraction.}$$

Proof. If $\mathcal{A}_{C_0}$ accepts a labelled tree, read off from every node its frontier profile, side-context descriptor, equality ports, pair table, and triple table. Conditions (C1)–(C8) are exactly the validity

clauses of a finite abstraction, and the parity condition supplies the required transitive vertical existential supports. Hence the accepted tree determines a valid finite abstraction.

Conversely, given a valid finite abstraction, label the regular unfolding by its frontier profiles. At each node choose the automaton transitions corresponding to the abstraction's declared child tuple, pair states, triple states, equality ports, and supports. Local validity makes every finite transition legal. Eventuality validity supplies accepting parity choices for all transitive requests. Thus the automaton has an accepting run. $\square$

Corollary 5.5 (Decidability). *Concept satisfiability for $\mathcal{ALCT}_{\text{RCC5}}$ over abstract strong RCC5 frames is decidable.*

Proof. Given C_0 , construct the finite automaton $\mathcal{A}_{C_0}$. Emptiness of finite alternating parity tree automata is decidable. By Theorem C, nonemptiness is equivalent to existence of a valid finite abstraction. By Theorem B, this is equivalent to existence of a saturated split-forest presentation satisfying C_0 . By Theorem A and the realization direction of Theorem B, this is equivalent to satisfiability of C_0 over abstract strong RCC5 frames. $\square$

6 Where the proof obligations live

The A/B/C separation makes the remaining proof obligations explicit.

Obligation	Discharged by
Witness-generated reduction	Witness thinning lemma in Section 3.1.
No semantic Hasse assumption	Generated cover presentation in Section 3.2.
Multiple selected superparts	Weak split occurrences and equality ports in Section 3.2.
Side/tree and side/side comparable witnesses	Ranked side contexts and forced verticalization in Section 3.3.
Residual DR/PO frontier	Residual-frontier lemma after saturation.
Blocking not equality	Explicit separation of $\equiv_{\text{blk}}$ from $\equiv_{\text{EQ}}$ and exact equality-port clauses.
Global RCC5 composition	Pair/triple product-state representatives and triple-state legality.
Universal propagation to nonselected targets	Pair-state exhaustion plus universal propagation clauses.
Transitive PP/PPI eventualities	Parity request monitors in Theorem C.
Effectivity	Finite closure, finite type set, bounded side rank, finite frontier positions, finite pair/triple states, finite automaton.

7 Finite bounds and effectivity

Let $n = |\text{Cl}(C_0)|$ and $t = |\text{Type}(C_0)| \leq 2^n$. Let $h(C_0)$ be the side-context rank bound. The number of abstract live positions is bounded by a computable expression of the form

$$K(C_0) \leq c \cdot n \cdot t \cdot h(C_0)$$

for a fixed construction-dependent constant c , after padding all missing children by dummy positions. A frontier profile is determined by:

- a subset of $K(C_0)$ positions;
- a type assignment to live positions;
- an equality-port equivalence relation;
- a relation matrix over live positions;
- finite side-context descriptors of rank at most $h(C_0)$;
- finite existential and universal support/obligation declarations.

Therefore the set of profiles is finite and effectively enumerable. Pair states are finite records over pairs of profiles, endpoint positions, relation labels, and finitely many interface-trace tags. Triple states are finite triples of pair states with overlap checks. Consequently $\Sigma(C_0)$, $\text{Pair}(C_0)$, $\text{Trip}(C_0)$, and $\mathcal{A}_{C_0}$ are all computable.

No useful elementary complexity bound is claimed here. The construction is intended as a decidability proof. Sharper complexity analysis would require a more economical normal form and is left separate.

8 Concluding formulation

The final decidability proof should be read as the composition of three independent modules:

Theorem A: split-forest normal form

Theorem B: finite abstraction adequacy

Theorem C: automaton correctness.

The split-forest abstraction survives as the key semantic enabler. What changes is the burden placed on it: it no longer has to implement cyclic blocking and simultaneous eventuality satisfaction by hand. Instead, it supplies a saturated, finite-interface tree-like normal form, and the parity automaton handles the regular infinite behaviour.

A Detailed side-context saturation argument

This appendix spells out the saturation process used in Theorem A. It is included because most defects in earlier split-forest drafts arose from unsaturated side interfaces.

Definition A.1 (Context closure operator). *For a source occurrence u , let $B(u)$ be the current bounded vertical boundary consisting of the current occurrence, declared equality mates, selected parent and child slots, and threshold positions for ancestor and descendant tails. Given a finite side diagram S , define $\Gamma_u(S)$ to be the least extension obtained by the following one-step operations:*

- (G1) *add each selected DR/PO witness of every occurrence already in S whose remaining side rank is positive;*
- (G2) *add the inherited label between every $s \in S$ and every $b \in B(u)$;*

- (G3) add the inherited label between every pair $s, t \in S$;
- (G4) if any such label is EQ, add an equality-port declaration and require equality congruence of types and external interfaces;
- (G5) if any such label is PP or PPI, add the corresponding local vertical stratum or threshold splice;
- (G6) if a new side witness is introduced, give it a subordinate side context of strictly smaller rank.

Lemma A.2 (Termination of saturation). *For fixed C_0 , iterating Γ_u terminates up to isomorphism within a computable bound.*

Proof. The rank of subordinate side calls strictly decreases. At any fixed rank, the construction uses only finitely many types, finitely many RCC5 labels, finitely many equality-port names, and a bounded number of selected existential requirements from $\text{Cl}(C_0)$. If two newly generated context positions have the same type, same relation vector to the boundary, same equality-port data, and same lower-rank context type, they are merged at the descriptor level. Hence at rank k only finitely many descriptor isomorphism classes can appear once all rank $< k$ descriptors are known. Induction on $k \leq h(C_0)$ gives a computable bound and termination up to descriptor isomorphism. $\square$

Lemma A.3 (Two-sided verticalization). *Let w be a side witness introduced for a source u , and let b be any represented vertical boundary occurrence of u , either on an ancestor trace or on a descendant trace. If the inherited label between b and w is PP or PPI, then the saturated context represents this interaction by a vertical stratum rather than by a residual side-frontier pair.*

Proof. The context closure operator inspects all side/tree pairs in both directions. If $b\text{PP}w$, then w is a proper superpart of b and an occurrence or equality mate of w is inserted into the ancestor-side threshold for b . If $b\text{PPI}w$, then w is a proper part of b and an occurrence or equality mate of w is inserted into the descendant-side threshold. These are the only two strict containment cases. The closure operator performs the insertion before residual-frontier classification, so the pair cannot remain residual. Infinite tails are represented by the vertical reachability/eventuality mechanism; the side context stores only the bounded threshold at which the tail enters the vertical backbone. $\square$

B Exact equality-port quotient

Definition B.1 (Typed equality congruence). *An equality-port relation $\equiv_{\text{EQ}}$ on a split presentation is a typed RCC5 congruence if, whenever $x \equiv_{\text{EQ}} y$:*

- (E1) $\text{tp}(x) = \text{tp}(y)$;
- (E2) for every occurrence z , the labels $\rho(x, z)$ and $\rho(y, z)$ agree after replacing z by its $\equiv_{\text{EQ}}$ -class;
- (E3) existential supports and universal obligations of x and y correspond under the same replacement;
- (E4) no strict vertical path connects x and y .

Lemma B.2 (Quotient soundness). *Let Φ be a weak split-forest presentation whose equality ports form a typed RCC5 congruence. The quotient $\Phi/\equiv_{\text{EQ}}$ is a strong RCC5 interpretation. Moreover, every closure formula has the same truth value at an occurrence and at its quotient class.*

Proof. Define the quotient domain as the set of $\equiv_{\text{EQ}}$ -classes. Put

$$\rho([x], [y]) = \rho(x, y).$$

This is well defined by the congruence condition. If $[x] = [y]$, exact equality ports give $\rho(x, y) = \text{EQ}$. If $[x] \neq [y]$, exactness forbids an EQ label between representatives, hence strong equality holds. Converse is inherited from the weak presentation. For composition, choose representatives x, y, z . Since Φ satisfies RCC5 composition,

$$\rho(x, z) \in \text{Comp}(\rho(x, y), \rho(y, z)).$$

Well-definedness transfers this statement to quotient classes.

Truth preservation is by induction on closure formulas. Atomic concepts and Boolean formulas follow from type equality on equality ports. Existentials are preserved because supports correspond under congruence. Universals are preserved because every quotient successor is represented by some occurrence successor, and universal obligations agree across equality classes. $\square$

C Pair and triple exhaustion

Definition C.1 (Interface distance). *For two occurrences x, y in an unfolded abstraction, the interface distance is the number of profile boundaries crossed by the unique path from x to y through their least common represented interface, counting equality-port and side-context transfers as one boundary step. Triples use the maximum of the three pair distances.*

Lemma C.2 (Pair-state exhaustion). *Every ordered pair of occurrences in an unfolding of a valid finite abstraction has a unique compatible pair-state representative.*

Proof. Induct on interface distance. Distance zero means both endpoints lie in the same live profile or saturated side context, so the local relation matrix supplies the pair state. For positive distance, move one endpoint one boundary step toward the least common represented interface. The induction hypothesis gives a pair state for the shortened pair. The product-state transition table composes that state with the crossed boundary label and returns the declared pair state for the original pair. Compatibility clauses ensure independence of equivalent choices through equality ports and side-context overlaps. $\square$

Lemma C.3 (Triple-state exhaustion). *Every ordered triple of occurrences in an unfolding of a valid finite abstraction has a compatible triple-state representative, and its three pair labels satisfy the RCC5 composition table.*

Proof. Apply pair-state exhaustion to the three ordered pairs. The product-state transition table is closed under every local triple and every cross-interface triple generated from child profiles, side contexts, equality-port transfers, and vertical paths. Therefore the three pair states determine a triple state in $\text{Trip}(C_0)$. Triple-state legality includes the condition

$$R_{13} \in \text{Comp}(R_{12}, R_{23}),$$

so the represented labels compose correctly. Since every concrete triple maps to such a representative, global RCC5 composition follows. $\square$

Lemma C.4 (Universal target exhaustion). *Let x be an occurrence carrying $\forall R.D$. If y is any occurrence with represented label $\rho(x, y) = R$, then the abstraction propagates D to the type of y .*

Proof. By pair-state exhaustion, (x, y) has a pair-state representative labelled R . The validity clauses require every universal obligation (x, R, D) to be propagated to every pair-state represented R -target, including selected witnesses, side-context witnesses, and targets arising only by composition through intermediate occurrences. Hence $D \in \text{tp}(y)$. $\square$

D Automaton finiteness and emptiness

The automaton used in Theorem C is finite for a simple reason: every object it stores is drawn from the finite closure and the finite abstraction alphabet.

Lemma D.1 (Effective construction). *From C_0 one can effectively enumerate the alphabet $\Sigma(C_0)$, the state set of $\mathcal{A}_{C_0}$, the transition relation, and the parity priority map.*

Proof. Compute $\text{Cl}(C_0)$ and the finite type set. Enumerate all bounded live position sets, all type assignments, all equivalence relations, all RCC5 relation matrices, all ranked side-context descriptors, all support declarations, and all universal obligation sets; discard those failing local legality. Enumerate pair states and triple states from the surviving profiles. Transitions are finite tables over parent profiles and bounded child-profile tuples. Priorities are assigned to the finitely many request-monitor states. Each step is a finite search. $\square$

Lemma D.2 (Emptiness is decidable). *It is decidable whether $L(\mathcal{A}_{C_0})$ is empty.*

Proof. Finite alternating parity tree automata have decidable emptiness, for example by the standard reduction to finite parity games. The game positions are pairs of automaton states and alphabet symbols, together with direction choices for the ranked input tree. The parity condition is inherited from the automaton. Since both the state set and alphabet are finite, the game graph is finite, and finite parity games are decidable. $\square$

E Suggested division between main paper and

A concise main decidability paper can now cite this as follows.

- (M1) State the abstract RCC5 semantics and closure definitions.
- (M2) State Theorem A and cite the normal-form proof in this .
- (M3) State Theorem B and cite the finite-abstraction adequacy proof in this .
- (M4) Give the automaton construction at a high level and cite Theorem C.
- (M5) Conclude decidability by automaton emptiness.

This keeps the main paper readable while leaving the fragile interface and equality-port details in a dedicated technical source.